

CPSC 375 High-Performance Computing

Lecture 29

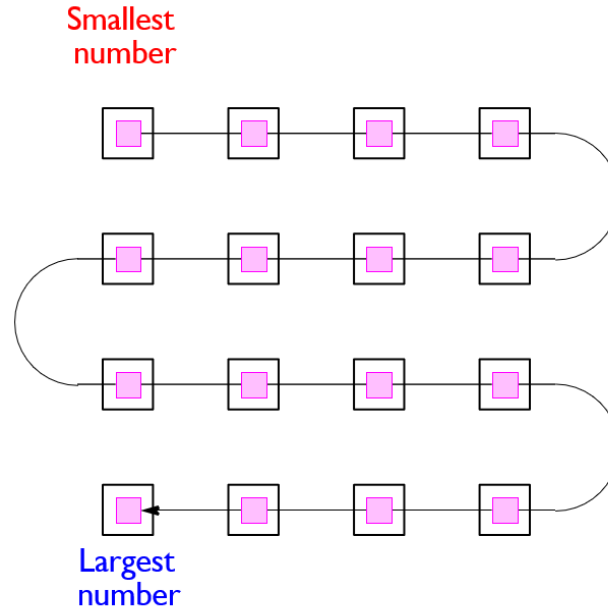
Parallel Sorting Algorithms III

Sorting on Specific Networks

- Algorithms can take advantage of the underlying interconnection network of the parallel computer.
- Two network structures have received specific attention: the *mesh* and *hypercube* because parallel computers have been built with these networks.
- Underlying architecture often hidden from user.
- MPI does provide features for mapping algorithms onto meshes, and one can always use a mesh or hypercube algorithm even if the underlying architecture is not the same.

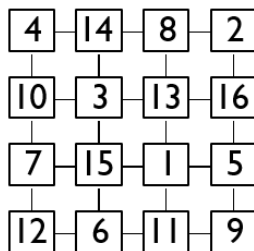
Two-Dimensional Sorting

- The layout of a sorted sequence on a mesh could be row by row or snakelike.

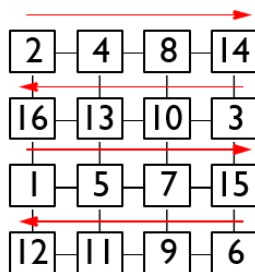


Shearsort

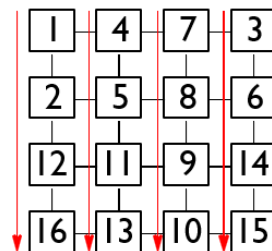
Alternate row and column sorting until list fully sorted. Row sorting alternative directions to get snake-like sorting:



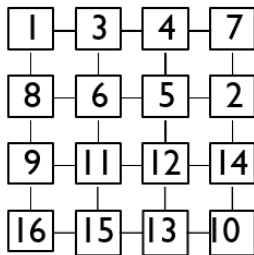
(a) Original placement of numbers



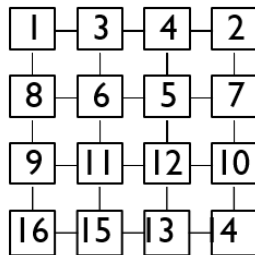
(b) Phase 1 — Row sort



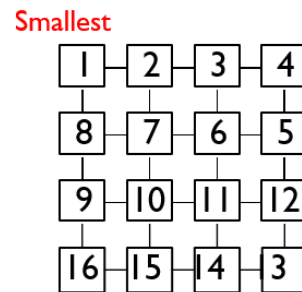
(c) Phase 2 — Column sort



(d) Phase 3 — Row sort



(e) Phase 4 — Column sort



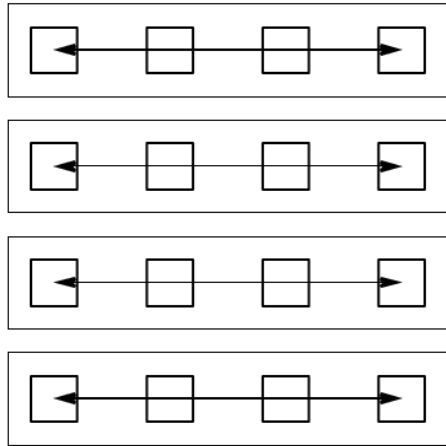
(f) Final phase — Row sort

Shearsort

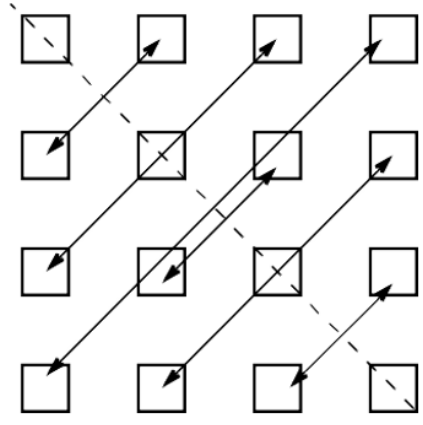
- Total steps: for an $N \times N$ mesh (n total elements), the algorithm converges in $\log N$ iterations.
- Time Complexity: Since each iteration involves sorting rows and columns of length N , the total parallel time is $O(N \log N)$ (or $O(\sqrt{n} \log \sqrt{n})$ in terms of total elements).
- Why use shearsort?
 - Simplicity
 - Easy to implement in hardware or parallel software because it only requires basic 1D sorting primitives.

Using Transposition

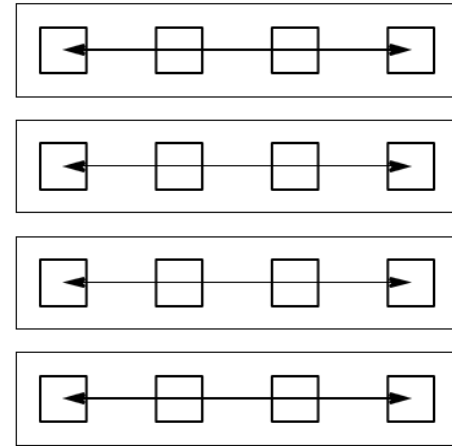
- Causes the elements in each column to be in positions in a row.
- Can be placed between the row operations and column operations:



(a) Operations between elements in rows



(b) Transpose operation



(c) Operations between elements in rows (originally columns)

Transposition in HPC

- Restricting communication to one dimension
 - Much faster (or easier to program) for a processor to talk to its neighbors in the same row than to reach across columns.
- Reducing complexity for limited processors
 - Each processor is typically assigned to one row.
- Cache locality and memory access
 - Improves spatial locality.

Other Sorting Algorithms

- We began by giving the lower bound for the time complexity of a sequential sorting algorithm based upon comparisons as $O(n \log n)$.
- Consequently, the time complexity of a parallel sorting algorithm based upon comparisons is $O((\log n)/p)$ with p processors or $O(\log n)$ with n processors.
- There are sorting algorithms that can achieve better than $O(n \log n)$ sequential time complexity and are very attractive candidates for parallelization, but they often assume special properties of the numbers being sorted.

Assignment 8: Radix Sorting

- Assumes numbers (or keys) to sort are represented in a positional digit representation, such as binary and decimal numbers.
- The digits represent values, and the position of each digit indicates its relative weighting.
- Strategy:
 - Radix sort starts with the least significant digit and sorts numbers according to their most significant digits.
 - The sequence is then sorted according to the next least significant digit and so on until the most significant digit, after which the sequence is sorted.
- For this to work, it is necessary that the order of numbers with the same digit is maintained, that is, one must use a *stable* sorting algorithm.

Example

- Unsorted: [170, 45, 75, 90, 802, 24, 2, 66]
- Pass 1:
Sort by: [170, 045, 075, 090, 802, 024, 002, 066]
Result: [170, 90, 802, 002, 024, 045, 075, 066]
- Pass 2:
Sort by: [170, 90, 802, 002, 024, 045, 075, 066]
Result: [802, 002, 024, 045, 066, 170, 075, 090]
- Pass 3:
Sort by: [802, 002, 024, 045, 066, 170, 075, 090]
Sorted: [002, 024, 045, 066, 075, 090, 170, 802]

Complexity of Serial Algorithm

- Time: $O(d(n + b))$ [homework]

where

d: number of digits (or passes).

n: number of elements.

b: the base of the number system (usually 10 for decimals)

- Space: $O(n + b)$ for the auxiliary arrays used in the *counting sort*.

Counting Sort

- Non-comparison-based sorting algorithm that operates by counting the number of objects that possess distinct key values.
- Uses the actual values of the elements as indices into an auxiliary array.
- It is particularly efficient when the range of input data is not significantly larger than the number of items to be sorted.

Counting Sort Algorithm

1. Create a count array where the index represents the value of the elements in the input.
2. Iterate through the input and increment the corresponding index in the count array for every instance of a value.
3. Transform the count array by adding each element to its predecessor (*prefix sum*).
 - Contains the actual position (index) of each element in the final sorted output.
4. Iterate through the original input array (from right to left to maintain stability) and place elements into a new output array based on the positions stored in the count array.

Example

- Unsorted: [1, 4, 1, 2, 5, 2]
- Step 1: Count frequencies
count: [0, 2, 2, 0, 1, 1]
- Step 2: Cumulative sum
csum: [0, 2, 4, 4, 5, 6]
- Step 3: Place the elements
(from right to left)
output: [1, 1, 2, 2, 4, 5]

See a 2: Look at index 2 in the csum (it says 4). Put 2 at position 4 and decrement the count to 3.

See a 5: Look at index 5 (it says 6). Put 5 at position 6 and decrement to 5.

...and so on.

Assignment: Parallel Radix Sort with MPI

- Implement a distributed-memory LSD Radix Sort in C using the MPI library to sort 32-bit unsigned integers.
- Utilize a *power-of-two radix* ($b=256$) and ensure stability across nodes by implementing global coordination with `MPI_Exscan` and `MPI_Alltoallv`.
- Develop and test your code on the Pine cluster for performance benchmarking.
- Measure and plot the strong scaling speedup as you increase the processor count, identifying the communication vs. computation bottleneck.

